



智能软件开发方向基础

第八章 聚类(clustering) --PART2. 聚类模型

张朝晖

2024.3~6



序号	内容
1	概述
2	机器学习的基本概念
3	模型的选择与性能评价
4	数据的获取、探索与准备
5	近邻模型-----分类、回归
6	决策树模型-----分类、回归
7	集成学习-----分类、回归
8	(朴素)贝叶斯模型-----分类
9	聚类
10	特征降维及低维可视化(PCA, t-SNE)
11	总复习

主要内容

1. 动态聚类

K-均值聚类(K-Means Clustering)

学习向量量化(LVQ)

2. 密度聚类

3. 层次聚类

4. 高斯混合模型



动态聚类的三个要点:

- ① 选定某种**距离度量**作为样本间的相异性度量
- ② 确定某个**准则函数**, 用于评价聚类结果的质量
- ③ 给定**初始划分方法**, 以迭代方式找出使**准则函数取值最优**的划分结果

动态聚类过程:

多次迭代,逐步调整类别划分, 最终使准则最优。

- C-Means Clustering
- Fuzzy C-Means Clustering
- ...



K - Means Clustering 聚类问题描述

输入: 样本集 $D = \{x_1, \dots, x_m\}$, $x_i = [x_{i1} \quad \dots \quad x_{id}]^T \in \mathbb{R}^d$
要生成的簇的数目 **K**

输出: **K**个互不相交的簇 $C = \{C_i \mid i = 1, \dots, K\}$

sklearn.cluster.KMeans

```
class sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++',
n_init='auto', max_iter=300, tol=0.0001, verbose=0, random_state=None,
copy_x=True, algorithm='lloyd')
```

[source]

1. 聚类准则--“最小误差平方和”准则

将样本集 D 划分成 k 簇: $D = C_1 \cup \dots \cup C_k$

簇的数目 k 样本数目 m

误差平方和目标函数 $E(\mu_1, \dots, \mu_k, C_1, \dots, C_k) = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$

其中 C_i --第 i 簇, $i = 1, \dots, k$

N_i --第 i 簇的样本数目

μ_i --第 i 簇的中心, $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$

注: $\{\mu_i, i = 1, \dots, k\}$ --codebook

$\sum_{x \in C_i} \|x - \mu_i\|^2$ -- 簇内样本关于该簇中心的距离平方和



2. 算法实现

初始化方式

交叉迭代的动态更新

$$\{\mu_i^{(j)}, i = 1, \dots, k\}$$

$$\rightarrow \{C_i^{(j+1)}, i = 1, \dots, k\}$$

$$\rightarrow \{\mu_i^{(j+1)}, i = 1, \dots, k\}$$

$\rightarrow \dots$

算法终止条件



输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$;
聚类簇数 k .

过程:

- 1: 从 D 中随机选择 k 个样本作为初始均值向量 $\{\mu_1, \mu_2, \dots, \mu_k\}$
- 2: repeat
- 3: 令 $C_i = \emptyset$ ($1 \leq i \leq k$)
- 4: for $j = 1, 2, \dots, m$ do
- 5: 计算样本 x_j 与各均值向量 μ_i ($1 \leq i \leq k$) 的距离: $d_{ji} = \|x_j - \mu_i\|_2$;
- 6: 根据距离最近的均值向量确定 x_j 的簇标记: $\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$;
- 7: 将样本 x_j 划入相应的簇: $C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$;
- 8: end for
- 9: for $i = 1, 2, \dots, k$ do
- 10: 计算新均值向量: $\mu'_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$;
- 11: if $\mu'_i \neq \mu_i$ then
- 12: 将当前均值向量 μ_i 更新为 μ'_i
- 13: else
- 14: 保持当前均值向量不变
- 15: end if
- 16: end for
- 17: until 当前均值向量均未更新

输出: 簇划分 $C = \{C_1, C_2, \dots, C_k\}$

> 初始化方式
> 交叉迭代的动态更新方式
> 迭代终止条件

A. 聚类中心的初始化

“代表点”选择

样本集 D 划分之前, 先选择代表点作为初始聚类核心;
再基于最近距离法, 产生各簇
聚类结果与初始代表点选择有关

几种“代表点”的选择方法:

-- 经验选择

-- “密度法”选择代表点

-- 随机选择代表点



K-Means ++ 中更为有效的聚类中心初始化算法

1. 聚类中心集合 M 的初始化: $M \leftarrow \emptyset$
2. 在样本集 D 内随机选择1个样本初始化聚类中心 μ_1 :
 $M \leftarrow M \cup \{\mu_1\}$
3. for $i = 2, \dots, K$ do
 - 3.1 对于样本集 D 内不同于 M 内 $(i-1)$ 个中心的每个样本 x ,
计算该样本关于 M 内 $(i-1)$ 个中心的距离平方最小值,
记为 $[d(x, M)]^2 = \min_{j=1, \dots, i-1} [d(x, \mu_j)]^2$
 - 3.2 按照概率 $\frac{[d(x^*, M)]^2}{\sum_{x_i \in D \setminus M} [d(x_i, M)]^2}$ 随机选择一个样本 x^* , 将其
作为初选的第 i 个聚类中心 μ_i
 - 3.3 $M \leftarrow M \cup \{\mu_i\}$
4. 返回 M

B. 簇的数目 k 的确定

通常要求事先给定“簇”数目 k .

若 k 未知, 可按如下方法确定

(a) 一般根据领域先验知识确定;

(b) 实验确定:

令 $k = 1, 2, 3, \dots$, 分别进行聚类, 得 $J_e(k)$,

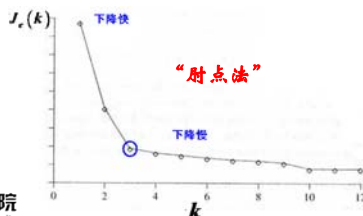
绘制 $J_e(k)$ - k 曲线图;

找出拐点, 对应聚类

数目为最终类别数。

该方法并不

总是有效。



C. 其它

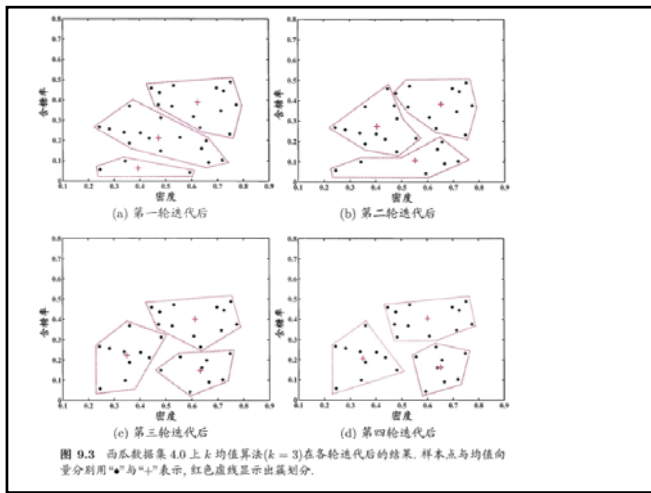
是否需要进行样本集的规范化预处理? --是

是否需要重复多次?

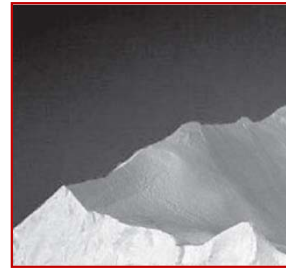
--从多个聚类结果中选择最好的那个

任意形状的聚类都可处理吗? --不





基于 K-均值聚类的图像分割



例：基于 K-means 聚类的颜色压缩(顺时针：原图，16 色，64 色，256 色)



主要内容

1. 动态聚类
 - K-均值聚类(K-Means Clustering)
 - 学习向量量化(LVQ)
2. 密度聚类
3. 层次聚类
4. 高斯混合模型

学习向量量化 (Learning Vector Quantization, LVQ)

问题描述

有标签的学习

输入：样本集 $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$

要生成的原型向量数目 q

其中 $x_i = [x_{i1} \ \dots \ x_{id}]^T \in X \subset R^d$
 $y_i \in Y$

输出：

- (1) q 个原型向量 $\{p_l \mid l=1, \dots, q\}$,
每个原型向量代表一个聚类簇，簇的标记值 $t_l \in Y$
- (2) 基于学得的原型向量，对原始空间进行划分

第一阶段 基于有标签样本集的原型向量的学习

找特征空间的代表点

注意：原型向量数目应
不低于标签值的数目

输入：样本集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;
原型向量个数 q , 各原型向量预设的类别标记 $\{t_1, t_2, \dots, t_q\}$;
学习率 $\eta \in (0, 1)$.

过程：

- 1: 初始化一组原型向量 $\{p_1, p_2, \dots, p_q\}$
 - 2: repeat
 - 3: 从样本集 D 随机选取样本 (x_j, y_j) ;
 - 4: 计算样本 x_j 与 p_i ($1 \leq i \leq q$) 的距离: $d_{ji} = \|x_j - p_i\|_2$;
 - 5: 找出与 x_j 距离最近的原型向量 p_{i^*} , $i^* = \arg \min_{i \in \{1, 2, \dots, q\}} d_{ji}$;
 - 6: if $y_j = t_{i^*}$ then
 - 7: $p' = p_{i^*} + \eta \cdot (x_j - p_{i^*})$
 - 8: else
 - 9: $p' = p_{i^*} - \eta \cdot (x_j - p_{i^*})$
 - 10: end if
 - 11: 将原型向量 p_{i^*} 更新为 p'
 - 12: until 满足停止条件
- 输出：原型向量 $\{p_1, p_2, \dots, p_q\}$ 比如：最大迭代次数

若 p_{i^*} 与 x_j 类别一致, 则 p_{i^*} 更新后为 $p' = p_{i^*} + \eta(x_j - p_{i^*})$

$$\begin{aligned}\|p' - x_j\|_2 &= \|p_{i^*} + \eta(x_j - p_{i^*}) - x_j\|_2 \\ &= \|(1-\eta)p_{i^*} - (1-\eta)x_j\|_2 \\ &= (1-\eta)\|p_{i^*} - x_j\|_2 < \|p_{i^*} - x_j\|_2\end{aligned}$$

若 p_{i^*} 与 x_j 类别不一致, 则 p_{i^*} 更新后为 $p' = p_{i^*} - \eta(x_j - p_{i^*})$

$$\begin{aligned}\|p' - x_j\|_2 &= \|p_{i^*} - \eta(x_j - p_{i^*}) - x_j\|_2 \\ &= \|(1+\eta)p_{i^*} - (1+\eta)x_j\|_2 \\ &= (1+\eta)\|p_{i^*} - x_j\|_2 > \|p_{i^*} - x_j\|_2\end{aligned}$$

第二阶段, 基于原型向量的样本空间X的簇划分。 --Voronoi Tessellation(V图剖分)

q 个原型向量 $\{p_l | l = 1, \dots, q\}$

$$\forall x \in X \subset \mathbb{R}^d$$

$$R_i = \{x \in X | \|x - p_i\|_2 \leq \|x - p_{i'}\|_2, i' \neq i\}$$

$$X = R_1 \cup R_2 \cup \dots \cup R_q$$

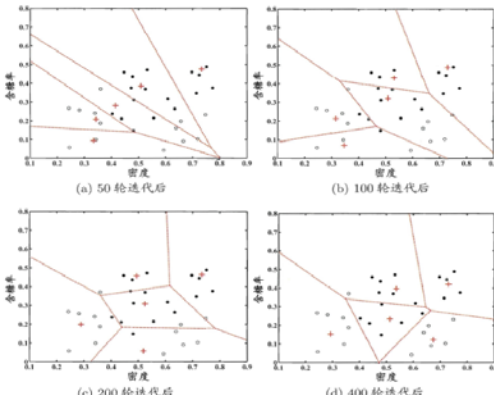


图 9.5 西瓜数据集 4.0 上 LVQ 算法($q=5$)在不同轮数迭代后的聚类结果。 c_1, c_2 类样本点与原型向量分别用“•”, “○”与“+”表示, 红色虚线显示出聚类形成的 Voronoi 剖分。

主要内容

1. 动态聚类

2. 密度聚类(DBSCAN)

Density - Based Spatial Clustering of Applications with Noise

3. 层次聚类

4. 高斯混合模型



```
class sklearn.cluster.DBSCAN(eps=0.5, *, min_samples=5,
metric='euclidean', metric_params=None, algorithm='auto', leaf_size=30,
p=None, n_jobs=None)
```

[source]

➤ DBSCAN 算法是一种基于高密度连通区域的、基于密度的聚类算法

➤ 能够将具有足够高密度的区域划分为簇

➤ 能够在具有噪声的数据中发现任意形状的簇

➤ 能够发现异常点



1. 有关概念[给定样本集 $D = \{x_1, \dots, x_m\}$]

[1] 两个全局邻域参数 ($\epsilon, MinPts$)

ϵ --邻域最大半径

$MinPts$ --给定样本的 ϵ -邻域内最小样本数。

[2] ϵ -邻域

对于 $\forall x_j \in D$, x_j 的 ϵ -邻域为 $N_\epsilon(x_j) = \{x_i \in D | dist(x_i, x_j) \leq \epsilon\}$

[3] 核心对象(core object)

若 $|N_\epsilon(x_j)| \geq MinPts$, 则称 x_j 为一个核心对象。

[4] 密度直达(directly density-reachable)

若 $x_j \in N_\epsilon(x_i)$, 并且 x_i 为一个核心对象, 则称 x_j 由 x_i 密度直达。

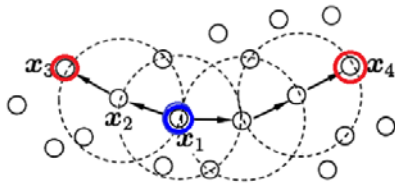
[5] 密度可达(density-reachable)

对于 x_i, x_j , 若存在样本序列 $p_1, p_2, \dots, p_n$, 其中 $p_1 = x_i, p_n = x_j$, 且 p_{i+1} 由 p_i 密度直达, 则称 x_j 由 x_i 密度可达。

[6] 密度相连 (density-connected)

对于 x_i 与 x_j , 若存在样本 x_k , 使得 x_i 与 x_j 均由 x_k 密度可达, 则称 x_i 与 x_j 密度相连。

邻域参数 ($\epsilon, MinPts$), ϵ -邻域, 核心对象, 密度直达, 密度可达, 密度相连



DBSCAN 定义的基本概念 ($MinPts = 3$): 虚线显示出 ϵ -邻域, x_1 是核心对象, x_2 由 x_1 密度直达, x_3 由 x_1 密度可达, x_3 与 x_4 密度相连。

[7] 边界对象 (border object)

若 $N_\epsilon(x_i) \geq MinPts$, $x_j \in N_\epsilon(x_i)$, 并且 $N_\epsilon(x_j) < MinPts$, 则称 x_j 为一个边界对象。

边界对象位于聚类簇的边界处。

[8] 噪声对象 (noise object)

核心对象、边界对象以外的其它样本。

[9] 簇

由密度可达关系导出的最大密度相连样本集。

给定邻域参数 ($\epsilon, MinPts$), 簇 $C \subseteq D$ 为满足以下性质的非空样本子集:

连接性 (connectivity): $x_i, x_j \in C \Rightarrow x_i$ 与 x_j 密度相连。

最大性 (maximality): $x_i \in C$, 并且 x_j 由 x_i 密度可达 $\Rightarrow x_j \in C$ 。

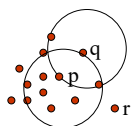


样本集 D 的主要组成

{ 核心对象 (聚类簇的种子点)
 边界对象 (可由核心对象密度可达的非核心对象)
 噪声对象 (异常点, 孤立点) } 聚类簇

高密度区

低密度区



$MinPts = 5$

$Eps = 1 \text{ cm}$



2. DBSCAN算法纲要

特点

-- 无需指定聚类簇的数目;

-- 只需输入两个全局参数 (全局参数取值如何选?)。

启发

-- 识别样本集内的核心对象;

-- 任选一个核心对象, 作为一个聚类簇的种子点;

-- 获取该种子点及其所有密度可达样本, 构成一个聚类簇。



➤ 遍历整个样本集 D , 确定核心对象、边界对象、及噪声对象

➤ 将所有噪声对象标记为异常。

➤ 考查每个核心对象: 核心对象彼此可密度直达, 则应划分至相同的聚类簇。

➤ 所有边界对象: 若它们是密度相连的, 应划分至与其核心对象一致的聚类簇内。



2. 算法描述

STEP1. 识别给定样本集 D 的所有核心对象。

得到核心对象集合 Ω

STEP2. 初始化聚类簇的数目为 0; 初始化未被访问的样本集为整个数据集 D 。

STEP3. 重复如下过程, 生成一系列聚类簇, 直到核心对象集合为空。

➤ 从核心对象集合中, 任选 1 核心对象, 作为聚类簇的一个种子点, 找出其密度可达的所有样本, 构成 1 个聚类簇。

➤ 更新核心对象集合;

➤ 更新未被访问的样本集合

STEP4. 输出所有聚类簇。

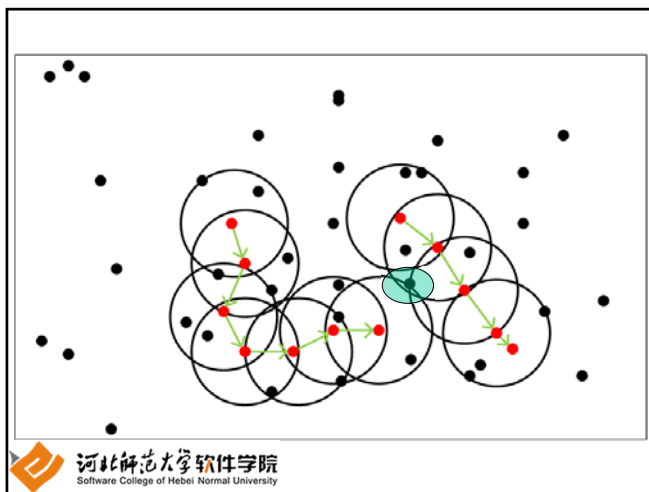
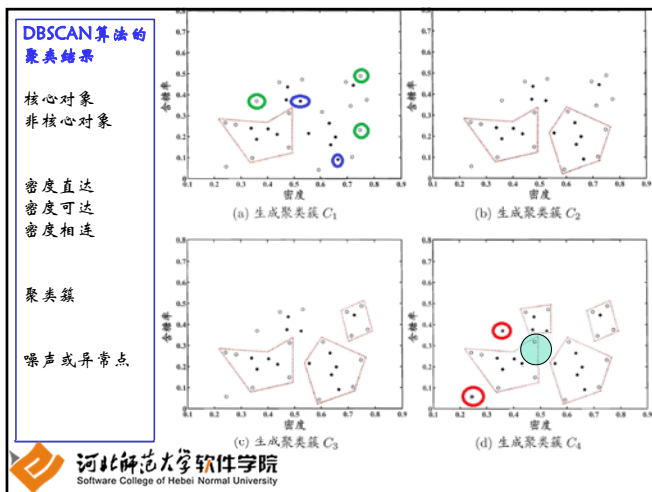
输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$;
邻域参数 ($\epsilon, MinPts$).

过程:

```

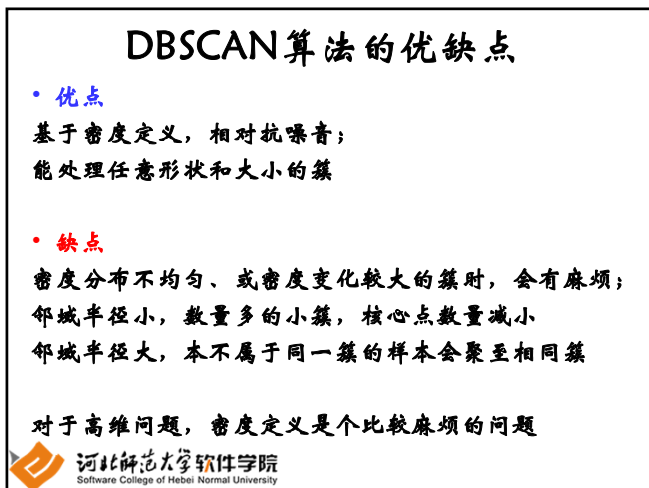
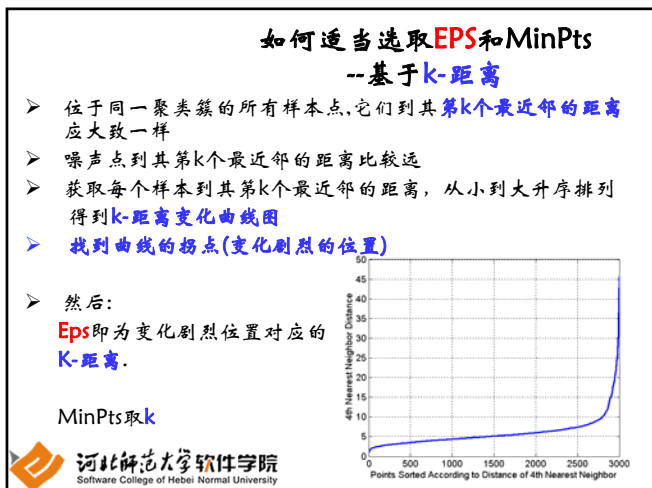
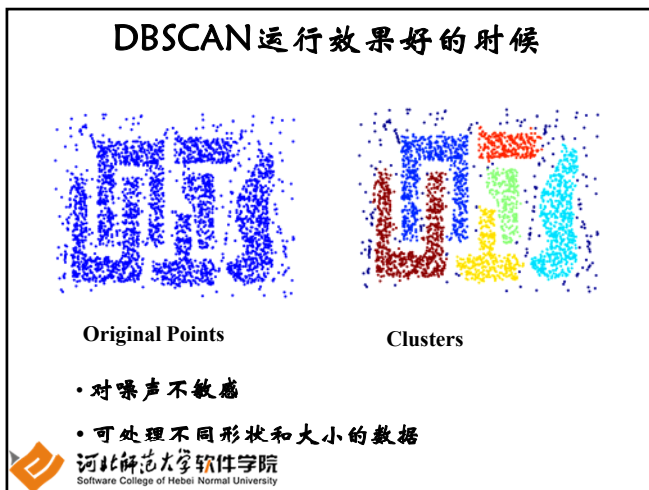
1: 初始化核心对象集合:  $\Omega = \emptyset$ 
2: for  $j = 1, 2, \dots, m$  do
3:   确定样本  $x_j$  的  $\epsilon$ -邻域  $N_\epsilon(x_j)$ ;
4:   if  $|N_\epsilon(x_j)| \geq MinPts$  then
5:     将样本  $x_j$  加入核心对象集合:  $\Omega = \Omega \cup \{x_j\}$ 
6:   end if
7: end for
8: 初始化聚类簇数:  $k = 0$ 
9: 初始化未被访问样本集合:  $\Gamma = D$ 
10: while  $\Gamma \neq \emptyset$  do
11:   记录当前未访问样本集合 ( $\Gamma_{old} = \Gamma$ )
12:   随机选取一个核心对象  $o \in \Omega$ , 初始化队列  $Q = \langle o \rangle$ ;
13:    $\Gamma = \Gamma \setminus \{o\}$  更新未访问的样本集合
14:   while  $Q \neq \emptyset$  do
15:     取出队列  $Q$  中的首个样本  $q$ ;
16:     if  $|N_\epsilon(q)| \geq MinPts$  then
17:        $\Delta = N_\epsilon(q) \cap \Gamma$ 
18:       将  $\Delta$  中的样本加入队列  $Q$ ;
19:        $\Gamma = \Gamma \setminus \Delta$  更新未访问的样本集合
20:     end if
21:   end while
22:    $k = k + 1$ , 生成聚类簇  $C_k = \Gamma_{old} \setminus \Gamma$ 
23:    $\Omega = \Omega \cup C_k$  更新核心对象集合
24: end while
  
```

输出: 簇划分 $C = \{C_1, C_2, \dots, C_k\}$



DBSCAN聚类算法的细节

河北师范大学软件学院
Software College of Hebei Normal University



主要内容

1. 聚类的引入

2. 动态聚类

3. 密度聚类(density-based clustering) DBSCAN

4. 层次聚类

hierarchical clustering

也称系统聚类、分级聚类

```
class sklearn.cluster.AgglomerativeClustering(n_clusters=2, *,
metric='euclidean', memory=None, connectivity=None,
compute_full_tree='auto', linkage='ward', distance_threshold=None,
compute_distances=False) [source]
```

1. 层次聚类的引入

(1) 问题描述

数据集 $D = \{x_1, \dots, x_m\} \Rightarrow$ 划分为合理的 k 个聚类簇。

$$1 \leq k \leq m$$

k 值的极端情况 $\begin{cases} \text{最多 } m \text{ 簇, 每簇包含一个样本} \\ \text{最少 } 1 \text{ 簇, 所有样本同属一簇} \end{cases}$



(2) 层次聚类的实现方式

样本数据的递归聚类：聚类过程中逐级考察簇间的距离，以此决定类别数。

合并式(聚合式)聚类(agglomerative clustering)

从聚类簇数目最多开始，**自底向上**，逐级合并距离最近的两个聚类簇，聚类簇的数目**逐渐减少**，直至与预设的聚类簇数目一致。经常使用。

如：**AGNES**算法 (AGglomerative NESTing)

分裂式聚类(divisive clustering)

从聚类簇数目最少开始，**自顶向下**，逐级分裂每级中最松散的聚类簇，聚类簇的数目逐级增加，直至与预设聚类簇数目一致。

如：**TSVQ**，可用于码本(codebook)的生成

(3) 层次聚类的几个关键问题

相似性(相异性)度量

聚类簇之间；
簇内样本之间

聚类簇合并 / 分裂停止的条件

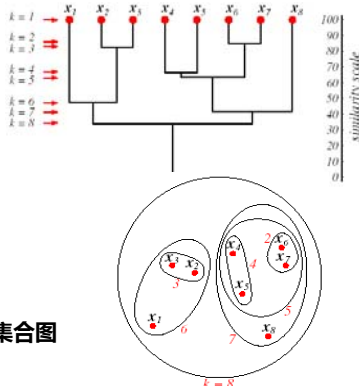
距离阈值；
预设的聚类簇数目

计算复杂度

(4) 聚合式系统聚类结果的表示

A. 树状图(dendrogram), 聚类树

可定量表示聚类簇间的相似性(或相异性)，簇间相似性标尺随着层数增加，逐渐减小。



B. 维恩图(venn diagram), 集合图

只能定性表示类间相似性

2. 聚合式系统聚类算法

给定样本集 $D = \{x_1, \dots, x_m\}$ ，选定簇间距离度量 $d(C_i, C_j)$ ，

将样本 D 聚成 k 簇。

不同的距离度量，可产生不同的聚类结果

STEP1 初始化。

A. 指定最终类别数 k ;

B. 初始划分 $\begin{cases} \text{每个样本自成1簇} & C_i \leftarrow \{x_i\} \quad i = 1, \dots, m \\ \text{初始类别数} & q \leftarrow m \end{cases}$

STEP2 合并。重复以下工作，直到 $q = k$ 。

A. 寻找最相似(距离最小)的两簇 $C_i, C_j (i < j)$

$$d(C_i, C_j) = \min_{\substack{\forall l, m \in \{1, \dots, q\} \\ \text{并且 } l < m}} d(C_l, C_m)$$

B. 记录最小距离，合并两类 C_i, C_j : $q \leftarrow q - 1$

STEP3 输出 k 个聚类簇。

AGNES算法描述

STEP1.初始化聚类簇

STEP2.初始化距离矩阵

STEP3.初始化聚类簇的数目

STEP4.连续合并最相似的两簇，直到满足规定的聚类簇数目：

- (1) 合并最相似的两簇；
- (2) 更新相关簇的序号；
- (3) 更新相关距离矩阵；
- (4) 更新簇的数目

STEP5.输出结果。

输入：样本集 $D = \{x_1, x_2, \dots, x_m\}$;
聚类簇距离度量函数 d ;
聚类簇数 k .

过程：

```

1: for  $j = 1, 2, \dots, m$  do
2:    $C_j = \{x_j\}$ 
3: end for
4: for  $i = 1, 2, \dots, m$  do
5:   for  $j = 1, 2, \dots, m$  do
6:      $M(i, j) = d(C_i, C_j)$ ;
7:      $M(j, i) = M(i, j)$ 
8:   end for
9: end for
10: 设置当前聚类簇个数:  $q = m$ 
11: while  $q > k$  do
12:   找出距离最近的两个聚类簇  $C_{i^*}$  和  $C_{j^*}$ ;
13:   合并  $C_{i^*}$  和  $C_{j^*}$ :  $C_{i^*} = C_{i^*} \cup C_{j^*}$ ;
14:   for  $j = j^* + 1, j^* + 2, \dots, q$  do
15:     将聚类簇  $C_j$  重编号为  $C_{j-1}$ 
16:   end for
17:   删除距离矩阵  $M$  的第  $j^*$  行与第  $j^*$  列;
18:   for  $j = 1, 2, \dots, q - 1$  do
19:     for  $i = 1, 2, \dots, q - 1$  do
20:        $M(i^*, j) = d(C_{i^*}, C_j)$ ;
21:        $M(j, i^*) = M(i^*, j)$ 
22:     end for
23:   end while
24:    $q = q - 1$ 
25: end while

```

输出：簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

3. 聚类簇 C_i, C_j 之间的连接 (linkage)

(1) 最小距离

$$d_{\min}(C_i, C_j) = \min_{\substack{x \in C_i \\ z \in C_j}} \text{dist}(x, z)$$

相应聚类算法就是“单链接”算法 (single linkage 或 single-link)

采用该距离度量聚类簇 C_i, C_j 之间的相异程度，进行聚类，就是产生最小生成树 (minimal spanning tree)

缺陷

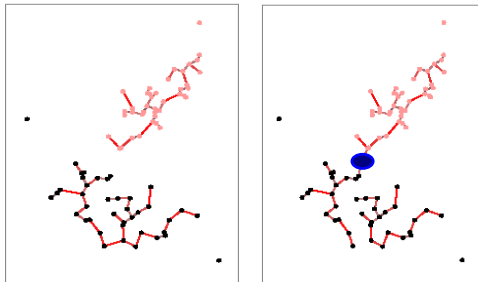
链接效应，产生细长的聚类；
聚类结果对噪声或数据点波动敏感。



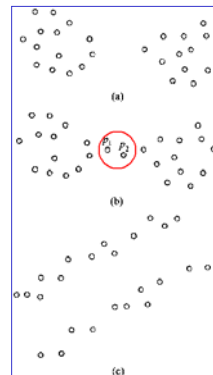
图. 二维高斯样本最小距离法层次聚类

左图 无干扰点

右图 有干扰点



最小距离法层次聚类 (single linkage 或 single-link)



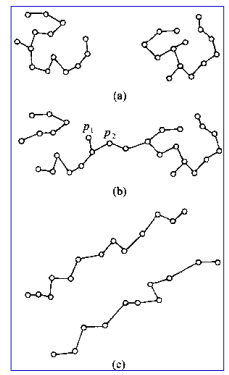
左图：

三种数据分布；

右图：

最近距离法的聚类结果

注意：a,b的区别



(2) 最大距离

$$d_{\max}(C_i, C_j) = \max_{x \in C_i, z \in C_j} \text{dist}(x, z)$$

最远邻算法

(the farthest-neighbor clustering algorithm)

全连接算法

(complete-linkage algorithm)

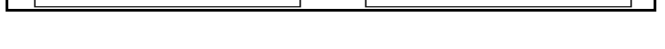
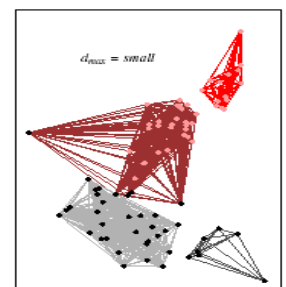
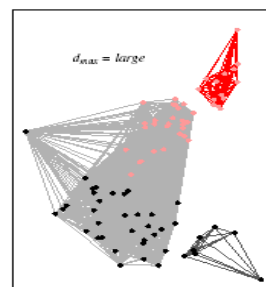
若 $d_{\max}(C_i, C_j) \geq d_0$ ，则聚类过程结束。



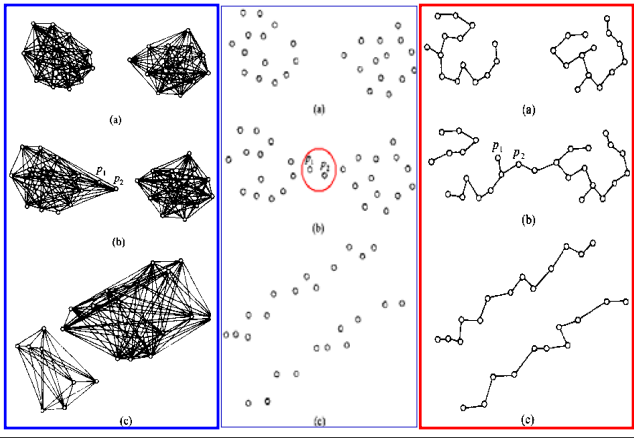
特点

防止两个密集点集通过某个路径聚为一簇的可能
不能检测出具有长条形状的聚类
适合紧密、体积相近的类别划分
聚类结果对个别远离点敏感

最大距离阈值 d_0 越大，聚类簇的数目越小。



最大距离法与最小距离法聚类结果比较



(3) 平均距离

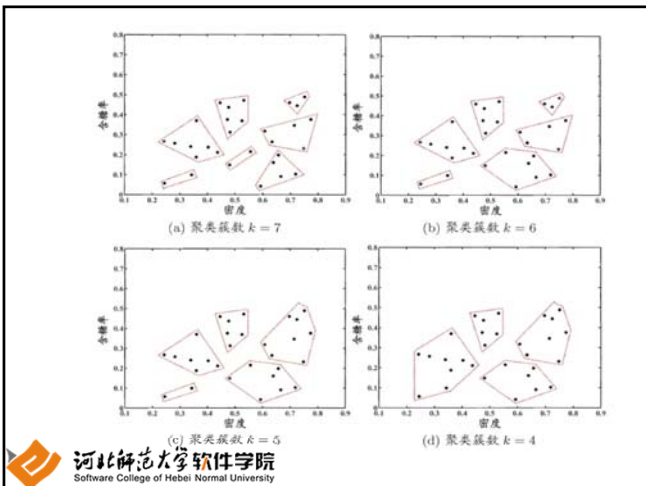
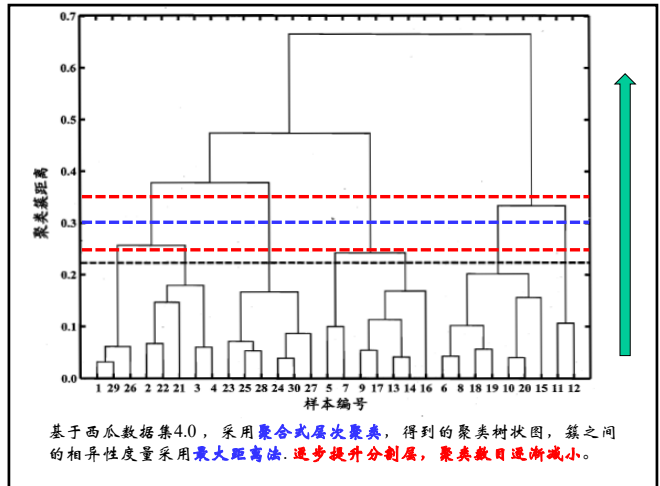
$$d_{avg}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{z \in C_j} dist(x, z)$$

相应的聚类算法称为“均链接”算法
(average-link, average linkage).

是关于前两种聚类算法的折中；计算简单。

西瓜数据集4.0

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459



思考题

- 什么是聚类？什么是分类？
请给出二者的区别与联系。
- 若采用不同模型对给定的数据集D进行划分，请给出不同聚类算法的实现步骤。
 - (1) k-均值聚类(目标函数形式、参数意义)
 - (2) 层次聚类(如何计算样本点之间距离、集合之间距离)
 - (3) DBSCAN聚类(几个术语、控制参数的意义)
- 上述聚类模型的适用场合。

主要内容

1. 聚类的引入
2. 动态聚类
3. 密度聚类(density-based clustering) DBSCAN
4. 层次聚类

hierarchical clustering

也称系统聚类、分级聚类

5. 基于模型的聚类

高斯混合聚类=高斯混合模型的学习+贝叶斯决策

Gaussian Mixture Model, GMM

-- 贝叶斯分类的扩展应用

```
class sklearn.mixture.GaussianMixture(n_components=1, *,
covariance_type='full', tol=0.001, reg_covar=1e-06, max_iter=100, n_init=1,
init_params='kmeans', weights_init=None, means_init=None,
precisions_init=None, random_state=None, warm_start=False, verbose=0,
verbose_interval=10) [source]
```

问题描述

输入：样本集 $D = \{x_1, \dots, x_m\}$, $x_i = [x_{i1} \dots x_{id}]^T \in R^d$
要生成的簇的数目 k

输出：以 k 个高斯模型的混合描述的簇 $C = \{C_l | l = 1, \dots, k\}$

混合高斯模型 $p_M(x) = \sum_{i=1}^k \alpha_i p(x | \mu_i, \Sigma_i)$

第 i 个高斯成分的概率密度函数(第 i 类的条件概率密度函数)

$$p(x | \mu_i, \Sigma_i) = \frac{1}{(2\pi)^{d/2} |\Sigma_i|^{1/2}} \exp \left[-\frac{1}{2} (x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i) \right]$$

高斯成分的混合系数：第 i 个高斯成分的先验概率 α_i $\sum_{j=1}^k \alpha_j = 1$

参数集合： $\{(\alpha_i, \mu_i, \Sigma_i) | i = 1, \dots, k\}$



D 中的每个观测样本按照相同的混合概率密度函数独立抽取，样本集 D 满足独立、同分布。

混合概率密度函数的形式已知，但参数未知。

利用样本集 D 估计已知形式的概率密度函数的未知参数，就叫做概率密度函数的参数估计问题。

由于 D 内每个样本都是无标签的，若采用无类标签的样本数据，进行概率密度函数的参数估计，就是概率密度函数的非监督式参数估计，常采用最大似然估计法。

然后利用估计得到的混合高斯模型，对数据集 D 划分。

高斯混合聚类的问题分解



STEP1. 高斯混合模型的非监督式参数估计。

基于样本集 $D = \{x_1, \dots, x_m\}$ 估计混合高斯模型的参数：

$$\alpha = \{\alpha_1, \dots, \alpha_k\}$$

$$\mu = \{\mu_1, \dots, \mu_k\} \quad \Sigma = \{\Sigma_1, \dots, \Sigma_k\}$$

$$p_M(x; \alpha, \mu, \Sigma) = \sum_{i=1}^k \alpha_i p(x | \mu_i, \Sigma_i)$$

STEP2. 基于最大后验概率的样本集划分。

$$P_M(z_j = i | x_j) = \frac{P(z_j = i) p(x_j | z_j = i)}{p_M(x_j)} = \frac{\alpha_i p(x_j | \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l p(x_j | \mu_l, \Sigma_l)}$$

$$\lambda_j = \arg \max_{i \in \{1, 2, \dots, k\}} P_M(z_j = i | x_j) \quad j = 1, \dots, m$$

1. 样本集 $D = \{x_1, \dots, x_m\}$ 的随机抽取



假设样本集 $D = \{x_1, \dots, x_m\}$ 中的各样本按照独立同分布的方式随机抽取得到。

$$p_M(x; \alpha, \mu, \Sigma) = \sum_{j=1}^k \alpha_j p(x | \mu_j, \Sigma_j)$$

抽取过程可以分解为：

for $i = 1, \dots, m$ 抽取样本 $x_i \in D$:

按照 $\alpha_1, \dots, \alpha_k$ 定义的先验分布，随机选择相应的高斯成分，其中 α_j 为第 j 个高斯成分出现的先验概率；

按照被选中成分的高斯分布，进行随机抽取得到观测样本 x_i 。

2. 似然函数及似然函数

(1) 观测样本 $x_i \in D$ 的似然值

$$p_M(x_i; \alpha, \mu, \Sigma) = \sum_{j=1}^k \alpha_j p(x_i | \mu_j, \Sigma_j)$$

(2) 样本集 D 的似然值 (likelihood, 或 n 个样本的联合似然值)

$D = \{x_1, \dots, x_m\}$ 内各样本独立抽取, 似然值:

$$L(D; \alpha, \mu, \Sigma) = \prod_{i=1}^m p_M(x_i; \alpha, \mu, \Sigma) = \prod_{i=1}^m \left\{ \sum_{j=1}^k \alpha_j p(x_i | \mu_j, \Sigma_j) \right\}$$

(3) 样本集 D 的对数似然值 (loglikelihood)

$$\begin{aligned} H(\alpha, \mu, \Sigma) &= LL(D; \alpha, \mu, \Sigma) = \ln L(D; \alpha, \mu, \Sigma) = \ln \left\{ \prod_{i=1}^m p_M(x_i; \alpha, \mu, \Sigma) \right\} \\ &= \sum_{i=1}^m \ln \{ p_M(x_i; \alpha, \mu, \Sigma) \} = \sum_{i=1}^m \ln \left\{ \sum_{j=1}^k \alpha_j p(x_i | \mu_j, \Sigma_j) \right\} \end{aligned}$$

3. 高斯混合模型的学习--最大似然估计

样本集 D 的对数似然值 (loglikelihood)

$$LL(D; \alpha, \mu, \Sigma) = \sum_{i=1}^m \ln \{ p_M(x_i; \alpha, \mu, \Sigma) \} = \sum_{i=1}^m \ln \left\{ \sum_{j=1}^k \alpha_j p(x_i | \mu_j, \Sigma_j) \right\}$$

因样本集 D 为给定信息, 对数似然值的大小取决于参数集合 α, μ, Σ , 因此称 $H(\alpha, \mu, \Sigma)$ 为基于样本集 D 的对数似然函数。

最大似然估计 (Maximum Likelihood Estimation, MLE):

$$\begin{aligned} \{\hat{\alpha}, \hat{\mu}, \hat{\Sigma}\} &= \arg \max_{\alpha, \mu, \Sigma} L(D; \alpha, \mu, \Sigma) = \arg \max_{\alpha, \mu, \Sigma} LL(D; \alpha, \mu, \Sigma) \\ &= \arg \max_{\alpha, \mu, \Sigma} \sum_{i=1}^m \ln \left\{ \sum_{j=1}^k \alpha_j p(x_i | \mu_j, \Sigma_j) \right\} \end{aligned}$$

最优化问题:

$$\begin{aligned} \max_{\alpha, \mu, \Sigma} & \sum_{i=1}^m \ln \left\{ \sum_{j=1}^k \alpha_j p(x_i | \mu_j, \Sigma_j) \right\} \\ \text{s.t. } & \alpha_i \geq 0 \quad \sum_{j=1}^k \alpha_j = 1 \end{aligned}$$



选择何种最优化方法估计上述目标函数的最优值解?
EM算法 (交叉迭代, “E步+M步”)

4. 最大似然估计的具体实现过程

$$\begin{aligned} \text{最优化问题} & \begin{cases} \max_{\alpha, \mu, \Sigma} \sum_{i=1}^m \ln \left\{ \sum_{j=1}^k \alpha_j p(x_i | \mu_j, \Sigma_j) \right\} \\ \text{s.t. } \alpha_i \geq 0 \quad \sum_{j=1}^k \alpha_j = 1 \end{cases} \end{aligned}$$

$$\begin{aligned} LL(D; \alpha, \mu, \Sigma) &= \sum_{i=1}^m \ln \left\{ \sum_{j=1}^k \alpha_j p(x_i | \mu_j, \Sigma_j) \right\} \\ &= \sum_{i=1}^m \ln \left\{ \sum_{j=1}^k \frac{\alpha_j}{(2\pi)^{d/2} |\Sigma_j|^{1/2}} \exp \left[-\frac{1}{2} (x_i - \mu_j)^T \Sigma_j^{-1} (x_i - \mu_j) \right] \right\} \end{aligned}$$

$$\nabla LL(D; \alpha, \mu, \Sigma) = ?$$



$$\frac{\partial}{\partial \mu_i} LL(D; \alpha, \mu, \Sigma) = ? \quad \frac{\partial}{\partial \Sigma_i} LL(D; \alpha, \mu, \Sigma) = ? \quad \frac{\partial}{\partial \alpha_i} LL(D; \alpha, \mu, \Sigma) = ?$$

$$(1) \quad \frac{\partial}{\partial \mu_i} LL(D; \alpha, \mu, \Sigma) = ?$$

$$\frac{\partial}{\partial \mu_i} LL(D; \alpha, \mu, \Sigma) = \sum_{j=1}^m P_M(z_j = i | x_j) \left[\Sigma_i^{-1} (x_j - \mu_i) \right]$$

$$\text{令 } \frac{\partial}{\partial \mu_i} LL(D; \alpha, \mu, \Sigma) = 0$$

$$\hat{\mu}_i = \frac{\sum_{j=1}^m P_M(z_j = i | x_j) x_j}{\sum_{j=1}^m P_M(z_j = i | x_j)} = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}} \quad i = 1, \dots, k$$

西瓜书209页式(9.34)

$$(2) \quad \frac{\partial}{\partial \Sigma_i} LL(D; \alpha, \mu, \Sigma) = ?$$

$$\frac{\partial}{\partial \Sigma_i} LL(D; \alpha, \mu, \Sigma) = \frac{1}{2} \sum_{j=1}^m P_M(z_j = i | x_j) \Sigma_i^{-1} \left[(x_j - \mu_i)(x_j - \mu_i)^T \Sigma_i^{-1} - I \right]$$

$$\text{令 } \frac{\partial}{\partial \Sigma_i} LL(D; \alpha, \mu, \Sigma) = 0$$

$$\text{则 } \sum_{j=1}^m P_M(z_j = i | x_j) \Sigma_i^{-1} (x_j - \mu_i)(x_j - \mu_i)^T \Sigma_i^{-1} = \sum_{j=1}^m P_M(z_j = i | x_j) \Sigma_i^{-1}$$



河北师范大学软件学院
 Software College of Hebei Normal University

$$\sum_{j=1}^m P_M(z_j = i | x_j) (x_j - \mu_i) (x_j - \mu_i)^T = \sum_{j=1}^m P_M(z_j = i | x_j)$$

所以 $\hat{\Sigma}_i = \frac{\sum_{j=1}^m P_M(z_j = i | x_j) (x_j - \mu_i) (x_j - \mu_i)^T}{\sum_{j=1}^m P_M(z_j = i | x_j)}$

即:

$$\hat{\Sigma}_i = \frac{\sum_{j=1}^m \gamma_{ji} (x_j - \hat{\mu}_i) (x_j - \hat{\mu}_i)^T}{\sum_{j=1}^m \gamma_{ji}} \quad i = 1, \dots, k$$

西瓜书209页式(9.35)



河北师范大学软件学院
 Software College of Hebei Normal University

(3) $\frac{\partial}{\partial \alpha_i} LL(D; \alpha, \mu, \Sigma) = ?$

固定 μ, Σ , 估计混合系数 α , 有:

$$\begin{cases} \max_{\alpha} \sum_{j=1}^m \ln \left\{ \sum_{i=1}^k \alpha_i p(x_j | \mu_i, \Sigma_i) \right\} \\ \text{s.t. } \alpha_i \geq 0 \quad \sum_{i=1}^k \alpha_i = 1 \end{cases}$$

$$\hat{\alpha}_i = \frac{1}{m} \sum_{j=1}^m P_M(z_j = i | x_j) = \frac{1}{m} \sum_{j=1}^m \gamma_{ji}$$

$i = 1, \dots, k$

西瓜书209页式(9.38)



河北师范大学软件学院
 Software College of Hebei Normal University

GMM模型参数估计的交叉迭代

混合高斯模型 $p_M(x) = \sum_{i=1}^k \alpha_i p(x | \mu_i, \Sigma_i)$

(1) 初始化 $\hat{\mu}_i(0), \hat{\Sigma}_i(0), \hat{\alpha}_i(0), i = 1, \dots, k$

(2) 交叉迭代直到满足终止条件:

E-STEP $\gamma_{ji}(n+1) = \frac{\alpha_i(n) p(x_j | \mu_i(n), \Sigma_i(n))}{\sum_{i=1}^k \alpha_i(n) p(x_j | \mu_i(n), \Sigma_i(n))}$ $\begin{cases} i = 1, \dots, k \\ j = 1, \dots, m \end{cases}$

$\gamma_{ji} = P_M(z_j = i | x_j)$

M-STEP $\begin{cases} \hat{\mu}_i(n+1) = \frac{\sum_{j=1}^m \gamma_{ji}(n+1) x_j}{\sum_{j=1}^m \gamma_{ji}(n+1)} \quad i = 1, \dots, k \\ \hat{\Sigma}_i(n+1) = \frac{\sum_{j=1}^m \gamma_{ji}(n+1) (x_j - \hat{\mu}_i(n+1)) (x_j - \hat{\mu}_i(n+1))^T}{\sum_{j=1}^m \gamma_{ji}(n+1)} \quad i = 1, \dots, k \\ \hat{\alpha}_i(n+1) = \frac{1}{m} \sum_{j=1}^m \gamma_{ji}(n+1) \quad i = 1, \dots, k \end{cases}$



河北师范大学软件学院
 Software College of Hebei Normal University

输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$;
高斯混合成分个数 k . **如何初始化GMM模型的参数?**

过程:

- 1: 初始化高斯混合分布的模型参数 $\{(\alpha_i, \mu_i, \Sigma_i) | 1 \leq i \leq k\}$
- 2: repeat
- 3: for $j = 1, 2, \dots, m$ do
- 4: 根据式(9.30)计算 x_j 由各混合成分生成的后验概率, 即 $\gamma_{ji} = p_M(z_j = i | x_j) (1 \leq i \leq k)$
- 5: end for
- 6: for $i = 1, 2, \dots, k$ do
- 7: 计算新均值向量: $\mu'_i = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}}$
- 8: 计算新协方差矩阵: $\Sigma'_i = \frac{\sum_{j=1}^m \gamma_{ji} (x_j - \mu'_i) (x_j - \mu'_i)^T}{\sum_{j=1}^m \gamma_{ji}}$
- 9: 计算新混合系数: $\alpha'_i = \frac{\sum_{j=1}^m \gamma_{ji}}{m}$
- 10: end for
- 11: 将模型参数 $\{(\alpha_i, \mu_i, \Sigma_i) | 1 \leq i \leq k\}$ 更新为 $\{(\alpha'_i, \mu'_i, \Sigma'_i) | 1 \leq i \leq k\}$
- 12: until 满足停止条件
- 13: $C_i = \emptyset (1 \leq i \leq k)$
- 14: for $j = 1, 2, \dots, m$ do
- 15: 根据式(9.31)确定 x_j 的簇标记 λ_j ;
- 16: 将 x_j 划入相应的簇: $C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$
- 17: end for

输出: 簇划分 $C = \{C_1, C_2, \dots, C_k\}$

PART1. 基于 EM 算法的 GMM 模型的学习

PART2. 根据学习得到的 GMM 模型, 基于贝叶斯决策, 生成数据集 D 的最终划分结果.



河北师范大学软件学院
 Software College of Hebei Normal University

思考题

1. 什么是高斯混合聚类模型?
2. 能正确表达高斯混合模型; 理解实现过程
3. 若已经完成高斯混合模型的学习, 如何实现给定数据集的划分?



河北师范大学软件学院
 Software College of Hebei Normal University

具体推导过程

 河北师范大学软件学院
Software College of Hebei Normal University

$$= \sum_{i=1}^m P_M(z_j = i | x_j) [\Sigma_i^{-1} (x_j - \mu_i)]$$

西瓜书209页式(9.34)

$$\sum_{i=1}^m P_M(z_j = i | x_j) (\mathbf{x}_j - \boldsymbol{\mu}_i) (\mathbf{x}_j - \boldsymbol{\mu}_i)^T = \sum_{i=1}^m \mathbf{\Sigma}_i P_M(z_j = i | x_j)$$

河北师范大学软件学院
Software College of Hebei Normal University

$$\sum_{j=1}^m P_M(z_j = i | x_j) (x_j - \mu_i) (x_j - \mu_i)^T = \sum_i \sum_{j=1}^m P_M(z_j = i | x_j)$$

所以 $\hat{\Sigma}_i = \frac{\sum_{j=1}^m P_M(z_j = i | x_j) (x_j - \mu_i) (x_j - \mu_i)^T}{\sum_{j=1}^m P_M(z_j = i | x_j)}$

即: $\hat{\Sigma}_i = \frac{\sum_{j=1}^m \gamma_{ji} (x_j - \hat{\mu}_i) (x_j - \hat{\mu}_i)^T}{\sum_{j=1}^m \gamma_{ji}} \quad i = 1, \dots, k$

西瓜书209页式(9.35)

河北师范大学软件学院
Software College of Hebei Normal University

(3) $\frac{\partial}{\partial \alpha_i} LL(D; \alpha, \mu, \Sigma) = ?$

固定 μ, Σ , 估计混合系数 α , 有:
$$\begin{cases} \max_{\alpha} \sum_{j=1}^m \ln \left\{ \sum_{i=1}^k \alpha_i p(x_j | \mu_i, \Sigma_i) \right\} \\ \text{s.t. } \alpha_i \geq 0 \quad \sum_{i=1}^k \alpha_i = 1 \end{cases}$$

构造Lagrange辅助函数

$$H(\alpha, \lambda) = \sum_{j=1}^m \ln \left\{ \sum_{i=1}^k \alpha_i p(x_j | \mu_i, \Sigma_i) \right\} + \lambda \left[\sum_{i=1}^k \alpha_i - 1 \right]$$

令 $\frac{\partial H(\alpha, \lambda)}{\partial \alpha_i} = \sum_{j=1}^m \frac{p(x_j | \mu_i, \Sigma_i)}{\sum_{i=1}^k \alpha_i p(x_j | \mu_i, \Sigma_i)} + \lambda = 0 \quad i = 1, \dots, k$

由贝叶斯公式知 $\sum_{j=1}^m \frac{P_M(z_j = i | x_j)}{\alpha_i} + \lambda = 0 \quad i = 1, \dots, k$

河北师范大学软件学院
Software College of Hebei Normal University

$$\sum_{j=1}^m \frac{P_M(z_j = i | x_j)}{\alpha_i} + \lambda = 0 \quad i = 1, \dots, k$$

则有 $\sum_{j=1}^m P_M(z_j = i | x_j) = -\alpha_i \lambda \quad i = 1, \dots, k$

从而有 $\sum_{i=1}^k \left[\sum_{j=1}^m P_M(z_j = i | x_j) \right] = \sum_{i=1}^k [-\alpha_i \lambda]$

$$\sum_{j=1}^m \sum_{i=1}^k P_M(z_j = i | x_j) = -\lambda \sum_{i=1}^k \alpha_i$$

由于 $\sum_{i=1}^k P_M(z_j = i | x_j) = 1, \sum_{i=1}^k \alpha_i = 1$ 所以 $m = -\lambda$

河北师范大学软件学院
Software College of Hebei Normal University

$$\sum_{j=1}^m P_M(z_j = i | x_j) = -\alpha_i \lambda \quad i = 1, \dots, k$$

$$m = -\lambda$$

$$\sum_{j=1}^m P_M(z_j = i | x_j) = \alpha_i m \quad i = 1, \dots, k$$

即 $\hat{\alpha}_i = \frac{1}{m} \sum_{j=1}^m P_M(z_j = i | x_j) = \frac{1}{m} \sum_{j=1}^m \gamma_{ji}$

$i = 1, \dots, k$

西瓜书209页式(9.38)